-LAPORAN PEMROGRAMAN BERBASIS OBJECT

JOBSHEET 9



Nama:Daffa Dwi Nugroho

NIM : 254107020140

Kelas: TI 2-A

Link Github: https://github.com/Dapa-yap/Praktikum-Berbasis-Objek.git

D-IV TEKNIK INFORMATIKA

JURUSAN TEKNOLOGI INFORMASI

TAHUN AJARAN 2026

JOBSHEET 9

Percobaan 1: Abstract Class (± 40 menit)

1. Pada package abstractclass, buat class Hewan sebagai abstract class berikut.

```java
package abstractclass;

public abstract class Hewan {
    protected int umur;

    protected Hewan() {
        this.umur = 0;
    }

    public void bertambahUmur() {
        this.umur += 1;
    }

    public abstract void bergerak();

}
```

2. Buat class Kucing yang meng-extend Hewan. Gunakan ikon lampu → Implement all abstract methods, lalu ubah isi method bergerak() menjadi seperti berikut.

```java
package abstractclass;

public class Kucing extends Hewan {

    @Override
    public void bergerak(){
        System.out.println("Berjalan dengan KAKI, \"tap..tap..\"");
    }

}
```

3. Dengan cara yang sama, buat class Ikan.

```java
package abstractclass;

public class Ikan extends Hewan {
    @Override
    public void bergerak(){
        System.out.println("Berenang dengan SIRIP, \"wush..wush..\"");
    }
}
```

4. Buat class biasa Orang. Class ini adalah pengguna dari abstract class Hewan.

```
actclass > Orang.java > Orang > peliharaanHewan(Hewan)
package abstractclass;

public class Orang {
    private String nama;
    private Hewan hewanPeliharaan;

    public Orang(String nama) {
        this.nama = nama;
    }

    public void peliharaanHewan(Hewan hewan) {
        this.hewanPeliharaan = hewan;
    }

    public void ajakPeliharaanJalanJalan(){
        System.out.println("Namaku " + this.nama);
        System.out.println("Hewan peliharaanku berjalan dengan cara: ");
        this.hewanPeliharaan.bergerak();
        System.out.println("----------------------------------------");
    }
}
```

5. Buat main class Program di package yang sama.

```
package abstractclass;

public class Program {
    Run | Debug
    public static void main(String[] args) {
        Kucing kucingKampung = new Kucing();
        Ikan lumbaLumba = new Ikan();

        Orang ani = new Orang(nama: "Ani");
        Orang budi = new Orang(nama: "Budi");

        ani.peliharaanHewan(kucingKampung);
        budi.peliharaanHewan(lumbaLumba);

        ani.ajakPeliharaanJalanJalan();
        budi.ajakPeliharaanJalanJalan();
    }
}
```

6. Jalankan class Program (klik kanan → Run File atau Shift + F6), lalu amati hasilnya.

```
class.Program
Namaku Ani
Hewan peliharaanku berjalan dengan cara:
Berjalan dengan KAKI, "tap..tap.."
----------------------------------------
Namaku Budi
Hewan peliharaanku berjalan dengan cara:
Berenang dengan SIRIP, "wush..wush.."
----------------------------------------
```

Pertanyaan Percobaan 1

Setelah setiap uji coba, kembalikan kode seperti semula sebelum mengerjakan nomor berikutnya.

1. Jelaskan jalannya program di atas. Mengapa pemanggilan this.hewanPeliharaan.bergerak() di class Orang menghasilkan output yang berbeda untuk Ani dan Budi?

- Karena method bergerak() di class Hewan merupakan abstract method yang implementasinya dibuat berbeda pada setiap class turunannya. Objek Ani menggunakan class Kucing, sedangkan objek Budi menggunakan class Ikan. Jadi, ketika method bergerak() dipanggil, program menjalankan method sesuai dengan jenis objek yang digunakan.

- Ani memelihara kucing, sehingga output-nya Berjalan dengan KAKI, "tap..tap..".
- Budi memelihara ikan, sehingga output-nya Berenang dengan SIRIP, "wush..wush..".

2. Tambahkan baris Hewan hewan = new Hewan(); pada method main(). Tunjukkan hasil kompilasinya dan jelaskan secara singkat.

```
package abstractclass;

public class Program {
    Run | Debug
    public static void main(String[] args) {
        Hewan hewan = new Hewan();
        Kucing kucingKampung = new Kucing();
        Ikan lumbaLumba = new Ikan();

        Orang ani = new Orang(nama: "Ani");
        Orang budi = new Orang(nama: "Budi");

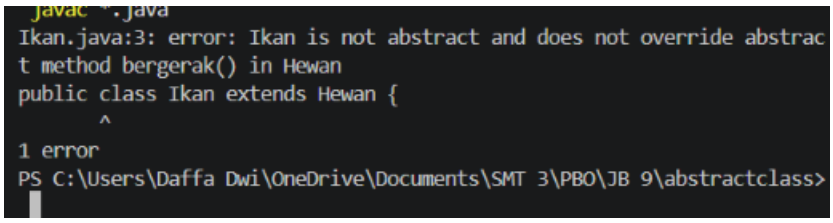
        ani.peliharaanHewan(kucingKampung);
        budi.peliharaanHewan(lumbaLumba);

        ani.ajakPeliharaanJalanJalan();
        budi.ajakPeliharaanJalanJalan();
    }
}
```

Program akan mengalami compile-time error karena class Hewan merupakan abstract class sehingga tidak dapat dibuat objek secara langsung menggunakan new.Alasannya, abstract class hanya berfungsi sebagai class induk yang akan diwarisi oleh class lain. Objek harus dibuat dari class konkret seperti Kucing atau Ikan.

3. Bolehkah sebuah class yang meng-extend abstract class tidak mengimplementasikan abstract method induknya? Buktikan dengan menghapus method bergerak() dari class Ikan, lalu tunjukkan hasil kompilasinya dan jelaskan.

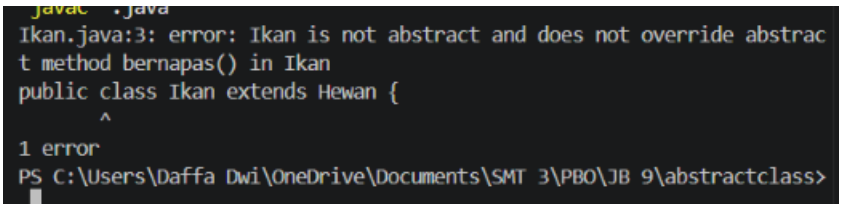
Boleh, tetapi class turunannya juga harus dideklarasikan sebagai abstract. Jika class Ikan tetap merupakan class biasa dan method bergerak() dihapus, program akan mengalami compile-time error karena belum mengimplementasikan method abstract dari class Hewan.



Jadi, jika Ikan ingin tetap menjadi class biasa, method bergerak() harus tetap dibuat dan di-override.

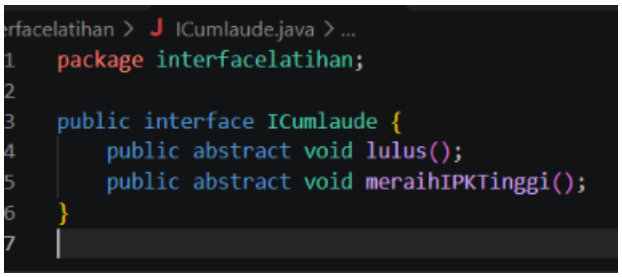
4. Tambahkan abstract method public abstract void bernapas(); di dalam class Ikan tanpa mengubah deklarasi class Ikan. Tunjukkan hasil kompilasinya dan jelaskan aturan yang dilanggar.

Penambahan method public abstract void bernapas(); pada class Ikan menyebabkan compile-time error. Hal ini terjadi karena class Ikan merupakan class biasa, tetapi memiliki abstract method bernapas() yang belum memiliki implementasi. Dalam Java, class yang memiliki abstract method harus dideklarasikan sebagai abstract class atau mengimplementasikan method tersebut.



Percobaan 2: Interface (± 45 menit)

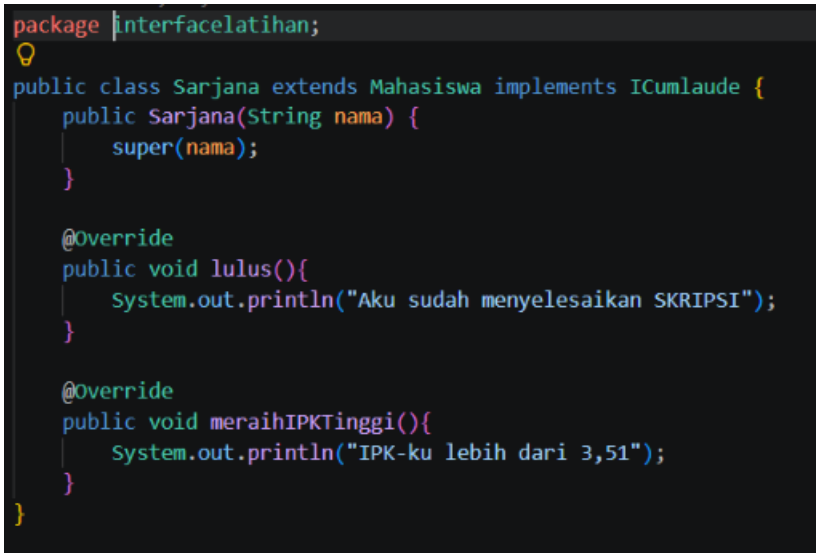
1. Pada package interfacelatihan, buat interface ICumlaude (klik kanan → New → Java Interface…) dengan dua abstract method: lulus() dan meraihIPKTinggi().



2. Buat class Mahasiswa.



3. Buat class Sarjana yang meng-extend Mahasiswa dan meng-implements ICumlaude. Gunakan ikon lampu untuk membuat kerangka method, lalu sesuaikan isinya.



4. Dengan cara yang sama, buat class PascaSarjana.

```java
package interfacelatihan;

public class PascaSarjana extends Mahasiswa implements ICumlaude {
    public PascaSarjana(String nama) {
        super(nama);
    }

    @Override
    public void lulus() {
        System.out.println("Aku sudah menyelesaikan TESIS");
    }

    @Override
    public void meraihIPKTinggi() {
        System.out.println("IPK-ku lebih dari 3,71");
    }

}
```

5. Buat class Rektor. Perhatikan bahwa parameter method beriSertifikatCumlaude() bertipe interface ICumlaude.

```java
package interfacelatihan;

public class Rektor {
    public void beriSertifikatCumlaude(ICumlaude mahasiswaCumlaude) {
        System.out.println("Saya REKTOR, memberikan sertifikat cumlaude");
        System.out.println("Selamat! Silakan perkenalkan diri Anda....");

        mahasiswaCumlaude.lulus();
        mahasiswaCumlaude.meraihIPKTinggi();

        System.out.println("----------------------------------------");
    }

}
```

6. Buat main class Program di package interfacelatihan.

```java
package interfacelatihan;

public class Program {
    Run | Debug
    public static void main(String[] args) {
        Rektor pakRektor = new Rektor();

        Mahasiswa mhsBiasa = new Mahasiswa(nama: "Charlie");
        Sarjana sarjanaCumlaude = new Sarjana(nama: "Dini");
        Sarjana masterCumlaude = new Sarjana(nama: "Elok");

        pakRektor.beriSertifikatCumlaude(mhsBiasa);
        pakRektor.beriSertifikatCumlaude(sarjanaCumlaude);
        pakRektor.beriSertifikatCumlaude(masterCumlaude);

    }
}
```

Pertanyaan Percobaan 2

1. Mengapa pada langkah 7 terjadi error? Tuliskan pesan error-nya dan jelaskan bagaimana Anda memperbaikinya.

```
Saya REKTOR, memberikan sertifikat cumlaude
Selamat! Silakan perkenalkan diri Anda....
Aku sudah menyelesaikan SKRIPSI
IPK-ku lebih dari 3,51
----------------------------------------
Saya REKTOR, memberikan sertifikat cumlaude
Selamat! Silakan perkenalkan diri Anda....
Aku sudah menyelesaikan SKRIPSI
IPK-ku lebih dari 3,51
----------------------------------------
```

Error terjadi karena objek mhsBiasa yang berasal dari class Mahasiswa dikirim sebagai argumen ke method beriSertifikatCumlaude(), padahal parameter method tersebut bertipe ICumlaude. Perbaikannya adalah menghapus baris berikut:

pakRektor.beriSertifikatCumlaude(mhsBiasa);

Sebab, class Mahasiswa tidak mengimplementasikan interface ICumlaude, sedangkan class Sarjana dan PascaSarjana sudah mengimplementasikannya.

2. Dapatkah method kuliahDiKampus() dipanggil dari objek sarjanaCumlaude di class Program? Mengapa demikian?

Bisa. Class Sarjana mewarisi class Mahasiswa menggunakan extends, sehingga method kuliahDiKampus() yang dimiliki class Mahasiswa dapat dipanggil melalui objek sarjanaCumlaude.

3. Dapatkah method kuliahDiKampus() dipanggil dari parameter mahasiswaCumlaude di method beriSertifikatCumlaude() pada class Rektor? Mengapa demikian?

Tidak bisa. Parameter mahasiswaCumlaude bertipe ICumlaude, sehingga method yang dapat dipanggil melalui parameter tersebut hanyalah method yang dideklarasikan dalam interface ICumlaude, yaitu lulus() dan meraihIPKTinggi()

4. Hapus kata kunci public pada method lulus() di class Sarjana. Tunjukkan hasil kompilasinya dan jelaskan, lalu kembalikan seperti semula.

```
Saya REKTOR, memberikan sertifikat cumlaude
Selamat! Silakan perkenalkan diri Anda....
Exception in thread "main" java.lang.IllegalAccessError: 'void interface
latihan.Sarjana.lulus()'
        at interfacelatihan.Rektor.beriSertifikatCumlaude(Rektor.java:8)
        at interfacelatihan.Program.main(Program.java:11)
PS C:\Users\Daffa Dwi\OneDrive\Documents\SMT 3\PBO\JB 9>
```

Program mengalami compile-time error karena method yang mengimplementasikan method interface harus memiliki akses public.

Percobaan 3: Multiple Interface (± 40 menit)

1. Pada package interfacelatihan, buat interface baru IBerprestasi.

```
package interfacelatihan;

public interface IBerprestasi {
    public abstract void menjualKompetisi();
    public abstract void membuatPublikasiIlmiah();


}
```

2. Ubah deklarasi class PascaSarjana dengan menambahkan IBerprestasi setelah kata kunci implements (dipisah koma). Gunakan ikon lampu untuk membuat kerangka method-nya, lalu tambahkan isi kedua method tersebut seperti berikut.

```
package interfacelatihan;

public class PascaSarjana extends Mahasiswa implements ICumlaude, IBerpres
    public PascaSarjana(String nama) {
        super(nama);
    }

    @Override
    public void lulus() {
        System.out.println("Aku sudah menyelesaikan TESIS");
    }

    @Override
    public void meraihIPKTinggi() {
        System.out.println("IPK-ku lebih dari 3,71");
    }
    @Override
    public void menjuaraiKompetisi() {
        System.out.println("saya telah menjuarai kompetisi INTERNASIONAL")
    }

    @Override
    public void membuatPublikasiIlmiah() {
        System.out.println("Saya telah menerbitkan artikel di jurnal INTER
    }

}
```

3. Tambahkan method beriSertifikatMawapres() pada class Rektor.

```java
package interfacelatihan;

public class Rektor {
    public void beriSertifikatCumlaude(ICumlaude mahasiswaCumlaude) {
        System.out.println("Saya REKTOR, memberikan sertifikat cumlaude");
        System.out.println("Selamat! Silakan perkenalkan diri Anda....");

        mahasiswaCumlaude.lulus();
        mahasiswaCumlaude.meraihIPKTinggi();

        System.out.println("----------------------------------------");
    }
    public void beriSertifikatMawapres(IBerprestasi mahasiswaBerprestasi)
        System.out.println("Saya REKTOR, memberikan sertifikat MAWAPRES");
        System.out.println("Selamat! Silakan perkenalkan diri Anda....");

        mahasiswaBerprestasi.menjuaraiKompetisi();
        mahasiswaBerprestasi.membuatPublikasiIlmiah();

        System.out.println("----------------------------------------");
    }
```

4. Ubah method main() pada class Program: jadikan komentar semua baris yang memanggil beriSertifikatCumlaude(), lalu beriSertifikatMawapres() seperti berikut.

```java
package interfacelatihan;

public class Program {
    Run | Debug
    public static void main(String[] args) {
        Rektor pakRektor = new Rektor();

        Mahasiswa mhsBiasa = new Mahasiswa(nama: "Charlie");
        Sarjana sarjanaCumlaude = new Sarjana(nama: "Dini");
        Sarjana masterCumlaude = new Sarjana(nama: "Elok");

        //pakRektor.beriSertifikatCumlaude(sarjanaCumlaude);
        //pakRektor.beriSertifikatCumlaude(masterCumlaude);

        pakRektor.beriSertifikatMawapres(sarjanaCumlaude);
        pakRektor.beriSertifikatMawapres(masterCumlaude);
    }
}
```

5. Akan terdapat error pada langkah 4 sehingga program tidak dapat dijalankan. Perbaiki kode program sehingga hasil eksekusinya sama seperti berikut.

```
PS C:\Users\Daffa Dwi\OneDrive\Documents\SMT 3\PBO\JB 9> java
atihan.Program
Saya REKTOR, memberikan sertifikat MAWAPRES
Selamat! Silakan perkenalkan diri Anda....
Aku berhasil menjuarai kompetisi
Aku berhasil membuat publikasi ilmiah
----------------------------------------
Saya REKTOR, memberikan sertifikat MAWAPRES
Selamat! Silakan perkenalkan diri Anda....
Aku berhasil menjuarai kompetisi
Aku berhasil membuat publikasi ilmiah
----------------------------------------
PS C:\Users\Daffa Dwi\OneDrive\Documents\SMT 3\PBO\JB 9>
```

Pertanyaan Percobaan 3

1. Pada interface IBerprestasi, ganti modifier public pada method menjadi protected dan pada method membuatPublikasiIlmiah() menjadi private. Apa yang terjadi? Jelaskan alasannya (sertakan capture hasilnya), lalu kembalikan seperti semula.

```java
package interfacelatihan;

public interface IBerprestasi {
    protected abstract void menjuaraiKompetisi();
    private abstract void membuatPublikasiIlmiah();


}
```

```
rfacelatihan\*.javawi\OneDrive\Documents\SMT 3\PBO\JB 9>
interfacelatihan\IBerprestasi.java:4: error: modifier protected not allo
wed here
    protected abstract void menjuaraiKompetisi();
                            ^
interfacelatihan\IBerprestasi.java:5: error: illegal combination of modi
fiers: abstract and private
    private abstract void membuatPublikasiIlmiah();
                          ^
```

- protected tidak boleh digunakan untuk method abstrak di dalam interface karena method interface yang abstrak harus bersifat public.
- private tidak bisa digunakan untuk method abstrak yang harus diimplementasikan oleh class. Method private pada interface harus memiliki implementasi sendiri.

2. Tambahkan baris IBerprestasi prestasi = new IBerprestasi(); pada method main(). Mengapa baris tersebut error?

```java
package interfacelatihan;

public class Program {
    public static void main(String[] args) {
        Iberprestasi prestasi = new Iberprestasi();
        Rektor pakRektor = new Rektor();

        Mahasiswa mhsBiasa = new Mahasiswa(nama: "Charlie");
        Sarjana sarjanaCumlaude = new Sarjana(nama: "Dini");
        Sarjana masterCumlaude = new Sarjana(nama: "Elok");

        //pakRektor.beriSertifikatCumlaude(sarjanaCumlaude);
        //pakRektor.beriSertifikatCumlaude(masterCumlaude);

        pakRektor.beriSertifikatMawapres(sarjanaCumlaude);
        pakRektor.beriSertifikatMawapres(masterCumlaude);
    }
}
```

Baris IBerprestasi prestasi = new IBerprestasi(); menghasilkan error karena interface tidak dapat diinstansiasi secara langsung. Untuk menggunakan interface, objek harus dibuat dari class konkret yang mengimplementasikan interface tersebut.

3. Apabila Sarjana Berprestasi harus menjuarai kompetisi NASIONAL dan menerbitkan artikel di jurnal NASIONAL, modifikasilah class-class yang terkait agar di class Program objek pakRektor dapat memberikan sertifikat mawapres kepada objek sarjanaCumlaude, dengan output seperti berikut.

```
Saya REKTOR, memberikan sertifikat MAWAPRES
Selamat! Bagaimana Anda bisa berprestasi?
Saya telah menjuarai kompetisi NASIONAL
Saya menerbitkan artikel di jurnal NASIONAL
----------------------------------------
Saya REKTOR, memberikan sertifikat MAWAPRES
Selamat! Bagaimana Anda bisa berprestasi?
Saya telah menjuarai kompetisi INTERNASIONAL
Saya menerbitkan artikel di jurnal INTERNASIONAL
----------------------------------------
```

```java
package interfacelatihan;

public class Program {
    Run | Debug
    public static void main(String[] args) {
        Rektor pakRektor = new Rektor();

        Mahasiswa mhsBiasa = new Mahasiswa(nama: "Charlie");
        Sarjana sarjanaCumlaude = new Sarjana(nama: "Dini");
        Magister masterCumlaude = new Magister(nama: "Elok");

        //pakRektor.beriSertifikatCumlaude(sarjanaCumlaude);
        //pakRektor.beriSertifikatCumlaude(masterCumlaude);

        pakRektor.beriSertifikatMawapres(sarjanaCumlaude);
        pakRektor.beriSertifikatMawapres(masterCumlaude);
    }
}
```

```java
package interfacelatihan;

public class Magister extends Mahasiswa implements IBerprestasi {

    public Magister(String nama) {
        super(nama);
    }

    @Override
    public void menjuaraiKompetisi() {
        System.out.println("Saya telah menjuarai kompetisi INTERNASIONAL");
    }

    @Override
    public void membuatPublikasiIlmiah() {
        System.out.println("Saya menerbitkan artikel di jurnal INTERNASIONA
    }
}
```

IV. Tugas (± 65 menit) Terdapat UML diagram dari sistem yang mengidentifikasi karakteristik dan jenis makanan beberapa binatang seperti gambar di bawah ini. Singa hanya makan daging sehingga tergolong Karnivora, sedangkan Keledai hanya mengonsumsi tumbuhan sehingga tergolong Herbivora. Gorilla dapat mengonsumsi kedua jenis makanan tersebut (omnivora), sehingga dikategorikan sebagai kombinasi Herbivora dan Karnivora. Setiap binatang juga memiliki karakteristik nama, suara, warna bulu, dan jumlah kaki.

```java
package Tugas;

public class Main {
    Run | Debug
    public static void main(String[] args) {
        Keledai keledai = new Keledai(nama: "Keledai", jmlKaki: 4, suara: "hehehe", warnaBulu: "Coklat");
        Singa singa = new Singa(nama: "Simba", jmlKaki: 4, suara: "Meroam", warnaBulu: "Coklat Muda");
        Gorilla gorilla = new Gorilla(nama: "Ucup", jmlKaki: 2,suara: "ngekngok" , warnaBulu: "Hitam Abu");

        System.out.println("Data Hewan");
        System.out.println("Keledai");
        keledai.displayData();
        System.out.println();

        System.out.println("Singa");
        singa.displayData();
        System.out.println();

        System.out.println("Gorilla");
        gorilla.displayData();
        }
    }
```

```java
package Tugas;

public class Singa extends Binatang implements IKarnivora{
    private String suara;
    private String warnaBulu;

    public Singa(String nama, int jmlKaki, String suara, String warnaBulu) {
        super(nama, jmlKaki);
        this.suara = suara;
        this.warnaBulu = warnaBulu;
    }

    @Override
    public void displayMakan() {
        System.out.println("Memakan Daging");
    }

    @Override
    public void displayBinatang() {
        System.out.println("Nama : " + nama);
        System.out.println("Jumlah Kaki : " + jmlKaki);
    }

    public void displayData() {
        this.displayBinatang();
        this.displayMakan();
        System.out.println("Suara :" + suara);
        System.out.println("Warna Bulu :" + warnaBulu);
    }
}
```

```java
package Tugas;

public class Keledai extends Binatang implements IHerbivora{
    private String suara;
    private String warnaBulu;

    public Keledai(String nama,int jmlKaki, String suara, String warnaBulu){
        super(nama, jmlKaki);
        this.suara = suara;
        this.warnaBulu = warnaBulu;
    }

    @Override
    public void displayMakan(){
        System.out.println("Keledai Pemakan Tumbuhan");
    }

    @Override
    public void displayBinatang(){
        super.displayBinatang();
    }

    public void displayData(){
        this.displayBinatang();
        this.displayMakan();
        System.out.println("Suara: "+suara);
        System.out.println("Warna Bulu: "+warnaBulu);
    }
}
```

```java
package Tugas;
public interface  IKarnivora {
   public void displayMakan();
}
```

```java
package Tugas;

public interface IHerbivora {
    public void displayMakan();
}
```

```java
package Tugas;

public class Gorilla extends Binatang implements IKarnivora, IHerbivora {
    private String suara;
    private String warnaBulu;

    public Gorilla(String nama, int jmlKaki, String suara, String warnaBulu) {
        super(nama, jmlKaki);
        this.suara = suara;
        this.warnaBulu = warnaBulu;
        this.jmlKaki = jmlKaki;
    }

    @Override
    public void displayMakan() {
        System.out.println("Gorilla memakan daging dan tumbuhan");
    }

    @Override
    public void displayBinatang() {
        super.displayBinatang();
    }

    public void displayData() {
        this.displayBinatang();
        this.displayMakan();
        System.out.println("Suara: " + suara);
        System.out.println("Warna Bulu: " + warnaBulu);
    }
}
```

```java
package Tugas;
public abstract  class Binatang {
    protected String nama;
    protected int jmlKaki;

    public Binatang(String nama, int jmlKaki) {
        this.nama = nama;
        this.jmlKaki = jmlKaki;
    }

    public void setNama(String nama) {
        this.nama = nama;
    }

    public String getNama() {
        return nama;
    }

    public void setKaki(int jmlKaki) {
        this.jmlKaki = jmlKaki;
    }

    public int  getKaki() {
        return jmlKaki;
    }

    public void displayBinatang(){
        System.out.println("Nama : " + nama + " Jumlah Kaki : " + jmlKaki);
    }
}
```

```
dt_ws\JB 9_97104b53\bin" Tugas.Main
Data Hewan
Keledai
Nama : Keledai Jumlah Kaki : 4
Keledai Pemakan Tumbuhan
Suara: hehehe
Warna Bulu: Coklat

Singa
Nama : Simba
Jumlah Kaki : 4
Memakan Daging
Suara :Meroam
Warna Bulu :Coklat Muda

Gorilla
Nama : Ucup Jumlah Kaki : 2
Gorilla memakan daging dan tumbuhan
Suara: ngekngok
Warna Bulu: Hitam Abu
PS C:\Users\Daffa Dwi\OneDrive\Documents\SMT 3\PBO\JB 9>
```